时钟之迹 课堂讲稿

STM32F407ZGT6的时钟配置与延时 彭俊杰 智装备241 2024306100324

讨论范围

这次沿着教师CH5EX01 HAL和STD的代码，回答四个问题：系统时钟在哪里建立，怎样改变，怎样读取，以及SysTick怎样形成延时。两份工程的板级接线相同，但HSE常量、初始化路径和滴答单位不同。另用普中SPL、独立HAL和CubeMX工程说明移植时容易出现的差异。

先确认工程身份

项目	教师HAL	教师STD
LED与流水灯	PE3、PE4；PB0至PB7	PE3、PE4；PB0至PB7
晶振假设	8 MHz	25 MHz
PLL参数	M8 N336 P2 Q4	M25 N336 P2 Q7
滴答周期	1 ms	10 ms
原件PE3完整周期	约2 s，即0.5 Hz	稳态约1 s，即1 Hz

教师HAL原件的Q支路超过规格，后文说明修正方法。原件数值按源文件原样保留以便逐行对应；修订值仅在修订版工程中生效，二者在档案中分别保存。

讲解时采用的证据

时钟数值与LED节拍由代码、课件和视频画面分别推导。验证边界统一界定如下：六份 AC5 历史日志为 0 错 0 警；仿真会话止步于 Flash 接口寄存器访问受限；PLL、GPIO 与延时波形属于待硬件实测的验证层级。

图解表示调用关系，代码框注明原件节选或伪代码。后续上板时，还要测量晶振、MCO或定时器输出，才能将“配置频率”与“实测频率”对应起来。

一 从复位走到时钟配置

复位后，处理器从向量表取得MSP初值和Reset_Handler入口。启动汇编调用SystemInit，再跳入Arm C运行库__main，完成数据初始化后进入main。函数名相同，并不保证两种库在SystemInit里做相同的事。

步骤	教师STD调用链	教师HAL调用链
启动汇编	Reset_Handler → SystemInit	Reset_Handler → SystemInit
SystemInit	设置FPU、系统配置并调用SetSysClock	设置FPU、系统配置及向量表位置
main前后	进入main时已尝试配置主PLL	进入main后HAL_Init → SystemClock_Config
外设与循环	RCC_GetClocksFreq → SysTick_Config → Delay(1) → LED_Init → 循环	MX_GPIO_Init → 循环

时钟初始化流程图解

```
Reset
  -> load initial MSP and Reset_Handler
  -> SystemInit
  -> __main: initialize C runtime
  -> main
       STD: use clock set by SetSysClock
       HAL: HAL_Init -> SystemClock_Config
  -> GPIO initialization
  -> application loop
```

HAL_Init先建立基础HAL环境和初始时基。SystemClock_Config调用HAL_RCC_OscConfig配置振荡器与PLL，再通过HAL_RCC_ClockConfig配置SYSCLK、AHB/APB分频和Flash等待周期。后者更新SystemCoreClock，并按新的HCLK重新建立HAL时基。

检查失败路径同样重要。教师STD在HSE启动失败后没有得到168 MHz，可能留在默认HSI 16 MHz；HAL原件则进入Error_Handler。任何等待就绪的循环都应考虑外部晶振缺失、参数越界和超时处理。

二 PLL与总线时钟的计算

系统时钟SYSCLK可选择HSI、HSE或PLL。以HAL原件的8 MHz输入为例，PLL先经M分频，再经N倍频，最后经P分频送入SYSCLK。

```
PLL input = HSE / M = 8 / 8 = 1 MHz
VCO       = PLL input * N = 1 * 336 = 336 MHz
SYSCLK    = VCO / P = 336 / 2 = 168 MHz
HCLK      = SYSCLK / 1 = 168 MHz
PCLK1     = HCLK / 4 = 42 MHz
PCLK2     = HCLK / 2 = 84 MHz
```

F407的HCLK、PCLK1、PCLK2最高分别为168、42、84 MHz。APB分频不为1时，相应定时器通常得到2倍PCLK，因此此例TIM2的内核时钟为84 MHz。GPIO 输出翻转速度由压摆率配置与指令执行开销决定，与 AHB 频率分属两类指标。

原件中必须单独处理的配置

教师HAL设置PLLQ=4，Q支路为336/4=84 MHz，超过DS8626 Rev12表36的75 MHz上限。独立修订版改为Q7后得到48 MHz，CPU与总线频率不变。即使没有调用USB，也不应把越界支路留作规范配置。

教师STD的25 MHz/M25同样得到1 MHz PLL输入。如果实际晶振为8 MHz而M仍为25，输入只有0.32 MHz，不满足PLL要求。53.76 MHz是形式计算值，不保证实际锁定；适配8 MHz板须同步改HSE_VALUE与PLL_M。

CubeMX视频中的频率设置

CubeMX新建工程视频10分20秒采用25 MHz、M20、N268、P2，所得主频为167.5 MHz。讲解接受这一接近值；它与上传clock168_gpio的8/M8/N336/P2属于两个不同配置版本，167.5 MHz 与 168 MHz 各自按其参数成立。

供电按芯片手册取VDD 1.8至3.6 V，并匹配稳压等级与Flash等待周期；开发板的 5 V 输入经板级稳压后供给MCU；VDD 供电范围按手册为 1.8–3.6 V，两者分属不同电源层级。

三 软件怎样获取当前频率

获取接口计算的是频率信息，并非用硬件仪器测量晶振。它们需要正确的RCC寄存器、HSE_VALUE/HSI_VALUE常量，以及由程序维护的软件缓存。

接口或变量	实际含义	检查重点
SystemCoreClock	HCLK的软件缓存	切频后是否更新
HAL_RCC_GetSysClockFreq	按RCC与振荡器常量推算SYSCLK	HSE_VALUE是否等于真实输入
HAL_RCC_GetHCLKFreq	直接返回SystemCoreClock	返回值为缓存结果，寄存器实际状态须另行读取
HAL_RCC_GetPCLK1Freq / PCLK2Freq	用HCLK缓存结合APB分频计算	缓存和分频都须正确
RCC_GetClocksFreq	标准库读取配置并填写频率结构	仍依赖时钟常量，不是实测

一个缓存过期的例子

普中时钟工程在SystemInit先配置168 MHz，随后main调用RCC_HSE_Config(8,336,4,7)，把PLLP改成4。实际配置对应84/21/42 MHz，但代码未调用SystemCoreClockUpdate，缓存仍可能保留168 MHz。如果直接用该旧值配置SysTick，等待时间也会出错。

```
/* After a valid direct RCC change: */
SystemCoreClockUpdate();
/* Then rebuild the timing base for the new HCLK. */
```

这段是使用顺序说明，并非对所有切频流程的完整替代。稳妥做法是走库提供的时钟配置流程，并检查返回值和RCC_CFGR.SWS；直接改寄存器还须保证安全的切换顺序、Flash等待周期与分频关系。

课堂提问：变量显示168000000是否证明芯片正在168 MHz运行？该数值存在两类非确认来源——复位默认值，或依赖错误 HSE 常量的计算结果。确认运行状态应同时核对最后一次有效配置、时钟源状态，并在条件具备时测量输出。

四 SysTick怎样产生滴答

SysTick是Cortex-M4内核中的24位递减定时器。选定计数时钟后，一个周期计数N次，LOAD应写N−1，周期为(LOAD+1)/fSysTick。CMSIS的SysTick_Config接收的是N，而不是已经减1的LOAD。

实现	计数时钟与周期	N与LOAD
教师HAL默认时基	HCLK168 MHz，1 ms	N168000；LOAD167999
教师CH5EX01 STD	HCLK168 MHz，10 ms	N1680000；LOAD1679999
若使用HCLK/8的1 ms周期	21 MHz，1 ms	N21000；LOAD20999

```
/* Standalone CMSIS setup, 168 MHz HCLK */
SysTick_Config(SystemCoreClock / 1000U);
/* CMSIS writes LOAD = ticks - 1. */

/* Teacher CH5EX01 STD uses a 10 ms tick */
RCC_GetClocksFreq(&RCC_Clocks);
SysTick_Config(RCC_Clocks.HCLK_Frequency / 100U);
```

HAL默认已由HAL_InitTick建立时基，main里不应为了调用HAL_Delay再初始化同一个SysTick。教师HAL的IOC存在HCLK/8文字，但实际CMSIS初始化设置CLKSOURCE=HCLK；分析运行逻辑时须沿生成源码与库函数追踪；IOC 中的一行文字不足以判定实际配置。

滴答单位由配置决定

教师CH5EX01 STD的Delay(1)对应一个10 ms tick，Delay(10)约100 ms。CH5EX02 STD改成HCLK/1000后，Delay的单位才变成约1 ms。函数名和旧注释没有办法替代这一步核对。

24位计数器限制单次重载范围。长延时应累计多个滴答；更改HCLK后必须重新计算时基。课件算例中的LOAD168000应纠正为167999。

五 Delay与HAL_Delay在等待什么

HAL等待累计毫秒数

SysTick_Handler调用HAL_IncTick，使uwTick随中断增长；HAL_GetTick返回这个累计值。HAL_Delay记录起点，用无符号时间差判断是否达到等待值。默认实现通常再加一个tick，以避免等待短于请求值。

```
/* HAL delay flow, simplified pseudocode */
start = HAL_GetTick();
wait = requested_ms + one_tick;
while ((uint32_t)(HAL_GetTick() - start) < wait) {
    /* SysTick IRQ advances uwTick. */
}
```

这仍是阻塞等待。如果关闭中断，或在SysTick无法抢占的中断处理程序中等待，uwTick可能不再增长，函数也无法退出。HAL_MAX_DELAY等边界处理应以完整库实现为准。

教师STD等待剩余滴答归零

```
/* Teacher STD delay flow, simplified pseudocode */
Delay(ticks):
    TimingDelay = ticks
    while TimingDelay != 0:
        wait

SysTick_Handler:
    if TimingDelay != 0:
        TimingDelay--
```

这份STD工程同样依赖周期中断，只是变量表示剩余滴答，CH5EX01每个滴答10 ms。它不同于普中SysTick例程：后者选择HCLK/8，重装LOAD、轮询COUNTFLAG，结束后关闭计数器。两种实现不可混称为同一个Delay。

更简单的while(i--)只是按指令空转，受主频、优化与中断影响。若把普中轮询版直接放进HAL工程并关闭SysTick，会破坏HAL延时与超时服务；可改用另一个定时器，而不是让两个模块同时占用SysTick。

六 把滴答换成LED节拍

1 Hz心跳指完整亮灭周期1 s，因此每隔约500 ms翻转一次输出。教师HAL原件1000 ms翻转对应约0.5 Hz；视频HAL版本100 ms推进流水、每5轮翻转PE3，对应约1 Hz。课件的“心跳”用数字翻转即可表达；真正渐亮渐暗的呼吸灯还需要PWM占空比变化。

推荐的独立节拍思路

```
/* Pseudocode: independent nominal intervals */
now = HAL_GetTick();
if ((uint32_t)(now - last_flow) >= 100U) {
    last_flow = now;
    advance_PB0_to_PB7();
}
if ((uint32_t)(now - last_heartbeat) >= 500U) {
    last_heartbeat = now;
    toggle_PE3();
}
```

以上是非阻塞调度思路，其工程化形态尚待烧录验证。循环足够及时运行时，两个输出各按自己的间隔推进；若任务长时间占用CPU，仍会推迟执行。对稳定采样或精确波形需求应采用硬件定时器；主循环轮询存在与指令执行相关的时基误差。

版本差异保留在原处

来源	等待与翻转	完整周期
课件例5-1要求	心跳1 Hz；PB依次循环	目标1 s
教师HAL RAR	1000 ms一次翻转	约2 s
教师STD RAR	100 ms一轮，每5轮翻转	稳态约1 s
HAL GPIO视频18分15秒	100 ms一轮，每5轮翻转	约1 s
SPL GPIO视频22分15秒	Delay20，每轮翻转；10 ms tick	约0.4 s

若用循环计数器每5步翻转，宜到5后显式归零，避免无符号计数器自然溢出影响模5分组。软件修改还需保持PB流水和PE3心跳需求独立，不要因修改某个Delay而同时改变所有节拍。

循环索引溢出的同余分析

教师视频例程采用单一 `uint16_t` 索引配合取模判别实现多速率分频：主循环每 100 ms 推进一次，索引在 65535 处回卷至 0。由于 $65535 = 5 \times 13107$ 且 $0 \equiv 0 \pmod 5$，回卷临界点前后相邻两次判别（相隔仅 100 ms）均满足翻转条件，标称 500 ms 的半周期在该处畸变为 100 ms 毛刺。规范实现采用 0–4 显式环形计数器，或基于 `HAL_GetTick()` 差值的毫秒调度；后者利用无符号减法的自然回卷特性，同时消除溢出毛刺与对全局变量的额外依赖。

七 构建证据与课堂小结

六份上传的Keil历史日志均为0 Error(s)、0 Warning(s)，采用Arm Compiler 5.06 update 7 build 960。Code大小依次为：普中时钟1524、普中SysTick1272、独立HAL3906、CubeMX3142、教师STD1528、教师HAL3390字节。Code=3906 的旧截图对应独立 HAL 目标，教师 HAL 目标为 3390 字节，二者分属不同构建记录。

现有Simulation截图显示独立HAL工程在HAL_Init的Flash缓存使能附近停下，error 65指向0x40023C00读权限。该记录表明仿真会话在 Flash 接口寄存器访问处受限，成因是仿真器 MMIO 虚拟化模型缺失，与实体硬件状态无关，亦不涉及 SysTick 内核功能的仿真完备性。本次没有取得GPIO波形、PLL锁定或延时误差的运行测量。

讲完后应能说明的四点

第一，沿最后一次有效调用定位实际时钟配置，区分STD在SystemInit中配置与HAL在main中配置。第二，PLL与总线计算要连同晶振、电压和Flash条件一起检查。第三，getter和SystemCoreClock给的是软件计算信息，正确性依赖常量与缓存维护。第四，先确认滴答单位和等待条件，再解释Delay参数与LED完整周期。

主要资料

教师CH5EX01 HAL原工程

教师CH5EX01 STD原工程

STM32F407ZG数据手册 DS8626

STM32F405和F407参考手册 RM0090

课程第5章GPIO与SysTick；Arm DUI0553B第4.4节；课程CubeMX新建工程、HAL GPIO、SPL GPIO视频。视频时间戳与原件版本见作业2的材料对照表。

CubeMX自定义逻辑应写在USER CODE BEGIN/END保护区内，避免重新生成时被覆盖；生成配置后仍须编写应用循环。下一次具备实物条件时，可输出MCO或定时器信号，测频并核对SysTick误差。